# Conteúdo

Projeto....2
Título....2
Objetivo....2
Autor, data, contexto do trabalho....2
Descrição do problema....2
Ferramentas e versões....3
Análise de dados....4
Análise de processos....4
Dicionário....5
String connection, instâncias, bases de dados....6
Lista de tabelas, campos e tipos....6
Tabela: Utilizador....6
Tabela: Peixe....6
Tabela: Captura....7
Tabela: Comentario....7
Namespaces....7
Classes e métodos....8
Outras informações....8

**Projeto**

**Título**

# FishCast

**Objetivo**

*Aplicação web de rede social que permite aos pescadores registar capturas e partilhar informação sobre a atividade de pesca local.*

**Autor, data, contexto do trabalho**

Lucas Gonçalves, 23 de março de 2026
Projeto Final de 50 horas, inserido no âmbito da formação em programação, com foco no desenvolvimento de uma aplicação web em ASP.NET para registo, partilha e análise de dados de pesca costeira local na zona do Cabedelo, em Viana do Castelo.

**Descrição do problema**

A pesca costeira local é uma prática cultural e de subsistência amplamente presente em comunidades litorâneas de Portugal, como Viana do Castelo, onde pescadores frequentam regularmente zonas como o Cabedelo e o Farol para a captura de espécies como o carapau, robálo e linguado. No entanto, a troca de informações entre pescadores sobre as condições do mar, o estado da maré, os períodos mais produtivos e os tipos de peixe que estão a ser apanhados ainda acontece de forma verbal e informal, sem qualquer registo organizado. Esta ausência de dados estruturados dificulta a tomada de decisões informadas sobre quando, onde e como pescar, especialmente para pescadores iniciantes que não dispõem da experiência acumulada dos mais velhos.

Não existem, no contexto local, aplicações digitais práticas que permitam registar capturas, partilhar observações sobre maré, horário e tipo de pesca, ou visualizar estatísticas simples baseadas em dados reais dos próprios pescadores. A ausência desta ferramenta impede que a comunidade piscatria aproveite o conhecimento coletivo acumulado ao longo do tempo, não sendo possível identificar padrões de captura por período do dia, região, condição de maré ou técnica utilizada (fundo ou superfície). Além disso, a falta de interação digital entre pescadores limita

a partilha de boas práticas e o acesso a informação útil por parte de quem está a iniciar-se na pesca.

O problema central é, portanto, a inexistência de uma solução digital acessível e centrada na comunidade que permita: registar capturas com dados como local, horário, maré e tipo de peixe; disponibilizar estatísticas simples sobre atividade de pesca por região; informar sobre os melhores períodos de pesca ao longo do dia; diferenciar padrões entre pesca de fundo e de superfície; facilitar a interação entre pescadores através da partilha de informações; e enviar notificações sobre atividade recente na zona. Para responder a esta necessidade, propõe-se o desenvolvimento da aplicação **FishCast**, uma aplicação web em ASP.NET Core MVC pensada para ser simples, funcional e acessível a qualquer utilizador, independentemente do seu nível de conhecimentos informáticos.

**Ferramentas e versões**

### Framework e Linguagem

**C#** — Linguagem de programação principal utilizada na implementação de toda a lógica do sistema.
**ASP.NET Core 8.0 (MVC)** — Framework principal para o desenvolvimento da aplicação Web, seguindo o padrão Model-View-Controller.

### Ambiente de Desenvolvimento

**Visual Studio 2022** — IDE utilizada para programar, compilar, depurar e executar o projeto, com suporte completo ao ASP.NET Core.
**Entity Framework Core 8** — ORM utilizado para gerir o acesso à base de dados de forma estruturada, através do mapeamento objeto-relacional.

### Base de Dados

**SQL Server 2019/2022** — Sistema de gestão de base de dados relacional utilizado para armazenar utilizadores, capturas, locais e comentários.
**SQL Server Management Studio (SSMS) 19** — Ferramenta gráfica para administração, consulta e gestão da base de dados SQL Server.

### Estilização e Interface

**Bootstrap 5** — Framework CSS para tornar a interface responsiva, organizada e acessível em qualquer dispositivo.
**HTML 5 e CSS** — Estrutura e estilização das páginas web da aplicação.
**JavaScript** — Utilizado para interações dinâmicas na interface, como filtros e validações no lado do cliente.

**Versionamento**

**Git** — Sistema de controlo de versões para gerir o histórico do código e acompanhar alterações ao longo do desenvolvimento.
**GitHub** — Plataforma de alojamento do repositório, garantindo segurança, organização e acesso remoto ao projeto.

## Análise de dados

O projeto **FishCast** tem como objetivo permitir o registo, partilha e consulta de dados de pesca costeira local, com foco inicial na zona do Cabedelo, em Viana do Castelo. O sistema foi pensado como uma aplicação web em **ASP.NET Core MVC**, com uma abordagem simples e colaborativa, funcionando como um "mini Instagram de pesca", onde os utilizadores podem publicar capturas, comentar e consultar padrões de pesca.

A análise de dados do sistema baseia-se em quatro entidades principais:

- **Utilizador** — representa quem utiliza o sistema, publica capturas e comenta.
- **Peixe** — representa os tipos de peixe registados nas capturas.
- **Captura** — entidade central do sistema, representando a publicação de pesca feita pelo utilizador.
- **Comentário** — permite a interação entre utilizadores nas publicações.

O sistema armazena dados como local de pesca, tipo de pesca, estado da maré, data e hora, quantidade capturada, peso, imagem e observações, permitindo posteriormente gerar estatísticas simples por peixe, horário, local e condições de captura.

## Análise de processos

O projeto **FishCast** tem como objetivo permitir o registo, partilha e consulta de dados de pesca costeira local, com foco inicial na zona do Cabedelo, em Viana do Castelo. O sistema foi pensado como uma aplicação web em **ASP.NET Core MVC**, com uma abordagem simples e colaborativa, funcionando como um "mini Instagram de pesca", onde os utilizadores podem publicar capturas, comentar e consultar padrões de pesca.

A análise de dados do sistema baseia-se em quatro entidades principais:

**Utilizador** — representa quem utiliza o sistema, publica capturas e comenta.

**Peixe** — representa os tipos de peixe registados nas capturas.

**Captura** — entidade central do sistema, representando a publicação de pesca feita pelo utilizador.

**Comentário** — permite a interação entre utilizadores nas publicações.

O sistema armazena dados como local de pesca, tipo de pesca, estado da maré, data e hora, quantidade capturada, peso, imagem e observações, permitindo posteriormente gerar estatísticas simples por peixe, horário, local e condições de captura.

Os principais processos do sistema são:

- **Registo e autenticação de utilizadores**
  O utilizador pode criar conta e iniciar sessão no sistema. O controlo da sessão será feito com variáveis de sessão, sem utilização do ASP.NET Identity nesta primeira versão.

- **Publicação de capturas**
  O utilizador autenticado pode registar uma nova captura, informando dados como local, peixe, tipo de pesca, maré, data/hora, quantidade, peso, observação e imagem.

- **Consulta de capturas**
  O sistema apresenta as capturas registadas sob a forma de feed/publicações, permitindo aos utilizadores visualizar a atividade recente.

- **Comentários nas capturas**
  Os utilizadores podem comentar publicações, promovendo a interação e partilha de informação entre pescadores.

- **Consulta de informação de apoio**
  O sistema permite consultar os peixes registados e, futuramente, gerar estatísticas simples com base nos dados acumulados.

## Dicionário

O sistema utiliza uma base de dados relacional para armazenar a informação principal do projeto.
Os dados são organizados em tabelas relacionadas entre si através de chaves primárias e chaves estrangeiras.

Os principais conceitos do dicionário de dados são:

- **PK (Primary Key)** — identificador único de cada registo.
- **FK (Foreign Key)** — chave estrangeira usada para relacionar tabelas.

- **Entidade** — representação de um objeto real do sistema, como Utilizador ou Captura.
- **Atributo** — característica de uma entidade, como Nome, Email ou Local.
- **Relacionamento** — associação entre entidades, como um utilizador ter várias capturas.

### String connection, instâncias, bases de dados

projeto utiliza **SQL Server** como sistema de gestão de base de dados.

**Base de dados:**
FishCastDB

**Tipo de instância:**
Instância remota de SQL Server acessível por endereço IP e porta específica.

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=62.28.39.135,62444;Database=FishCastDB;user=efa0125;password=efa0125@;TrustServerCertificate=True;"  // minha string de conexão do SQL Server
}
```

Esta string de ligação permite ao sistema ASP.NET Core conectar-se à base de dados através do Entity Framework Core.

### Lista de tabelas, campos e tipos

| Tabela: Utilizador | | |
|---|---|---|
| **Campos** | **Tipos** | **Tipos (DB)** |
| **Id** | int | chave primária (PK - Identity) |
| **Nome** | string | varchar(100) |
| **Email** | string | varchar(150) (unique) |
| **Password** | string | varchar(255) |
| **IsAdmin** | bool | |
| **FotoPerfil** | string | varchar(255) |
| **DataCriacao** | DateTime | datetime |

| Tabela: Peixe | | |
|---|---|---|
| **Campos** | **Tipos** | **Tipos (DB)** |
| **Id** | int | chave primária (PK - Identity) |
| **Nome** | string | varchar(100) |
| **Tipo** | string | varchar(50) |
| **Especie** | string | varchar(100) |
| **TamanhoCm** | int | int |

| Tabela: Captura | | |
|---|---|---|
| **Campos** | **Tipos** | **Tipos (DB)** |
| **Id** | int | chave primária (PK - Identity) |
| **UtilizadorId** | int | chave estrangeira (FK) |
| **PeixeId** | int | chave estrangeira (FK) |
| **Local** | string | varchar(150) |
| **TipoPesca** | string | varchar(50) |
| **Mare** | string | varchar(20) |
| **DataHora** | DateTime | datetime |
| **Quantidade** | int | int |
| **PesoKg** | decimal | Decimal(5,2) |
| **Observacao** | string | varchar(255) |
| **Imagem** | string | varchar(255) |

| Tabela: Comentario | | |
|---|---|---|
| **Campos** | **Tipos** | **Tipos (DB)** |
| **Id** | int | chave primária (PK - Identity) |
| **CapturaId** | int | chave estrangeira (FK) |
| **UtilizadorId** | int | chave estrangeira (FK) |
| **Texto** | string | varchar(255) |
| **DataComentario** | DateTime | datetime |

## Namespaces

# FishCast

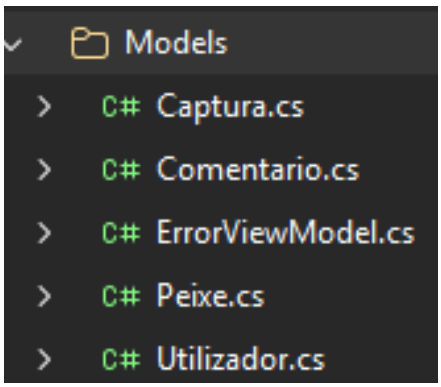
*Projeto ASP.NET Core 10.0 (MVC) | Lucas Goncalves | 2026*

Os principais namespaces do projeto FishCast poderão ser organizados da seguinte forma:

- **FishCast.Controllers**
- **FishCast.Data**
- **FishCast.Models**
- **FishCast.Services**
- **FishCast.Views**

**Classes e métodos**

## Classes Models




## Classe ApplicationUser

**Objetivo:**
Representar o utilizador do sistema, sendo responsável pela autenticação, identificação e interação na aplicação, utilizando o mecanismo ASP.NET Identity para gestão segura de contas.

**Principais métodos (operações do sistema):**

- RegistarUtilizador() — permite criar uma nova conta
- FazerLogin() — valida credenciais e inicia sessão
- TerminarSessao() — encerra a sessão do utilizador
- EditarPerfil() — permite atualizar dados do utilizador

```
using Microsoft.AspNetCore.Identity;

namespace FishCast.Models
{
    0 references
    public class ApplicationUser : IdentityUser
    {
        0 references
        public string Nome { get; set; }

        0 references
        public string? FotoPerfil { get; set; }

        0 references
        public DateTime DataCriacao { get; set; }
    }
}
```

## Classe Peixe

**Objetivo:**
Representar os tipos de peixe disponíveis no sistema, permitindo padronizar a informação associada às capturas.

**Principais métodos:**

- AdicionarPeixe() — regista um novo tipo de peixe
- ListarPeixes() — apresenta os peixes disponíveis
- EditarPeixe() — atualiza informações de um peixe
- RemoverPeixe() — remove um peixe do sistema

```
1   namespace FishCast.Models
2   {
        0 references
3       public class Peixe
4       {
            0 references
5           public int Id { get; set; }
6
            0 references
7           public string? Nome { get; set; }
8
            0 references
9           public string? Tipo { get; set; }
10
            0 references
11          public string? Especie { get; set; }
12
            0 references
13          public int? TamanhoMedioCm { get; set; }
14      }
15  }
```

## Classe Captura

**Objetivo:**
Representar uma captura de pesca, sendo a entidade central do sistema e funcionando como uma publicação na rede social de pescadores.

**Principais métodos:**

- CriarCaptura() — regista uma nova captura
- ListarCapturas() — apresenta as capturas registadas (feed)
- DetalharCaptura() — mostra os detalhes de uma captura
- EditarCaptura() — permite alterar dados da captura
- RemoverCaptura() — elimina uma captura

```
namespace FishCast.Models
{
    0 references
    public class Captura
    {
        0 references
        public int Id { get; set; }

        0 references
        public int UtilizadorId { get; set; }

        0 references
        public int PeixeId { get; set; }

        0 references
        public string? Local { get; set; }

        0 references
        public string? TipoPesca { get; set; }

        0 references
        public string?   Mare { get; set; }

        0 references
        public DateTime DataHora { get; set; }

        0 references
        public int Quantidade { get; set; }

        0 references
        public decimal? PesoKg { get; set; }

        0 references
        public string? Observacao { get; set; }

        0 references
        public string? ImagemPath { get; set; }
    }
}
```

## Classe `Comentario`

**Objetivo:**
Permitir a interação entre utilizadores através de comentários associados às capturas.

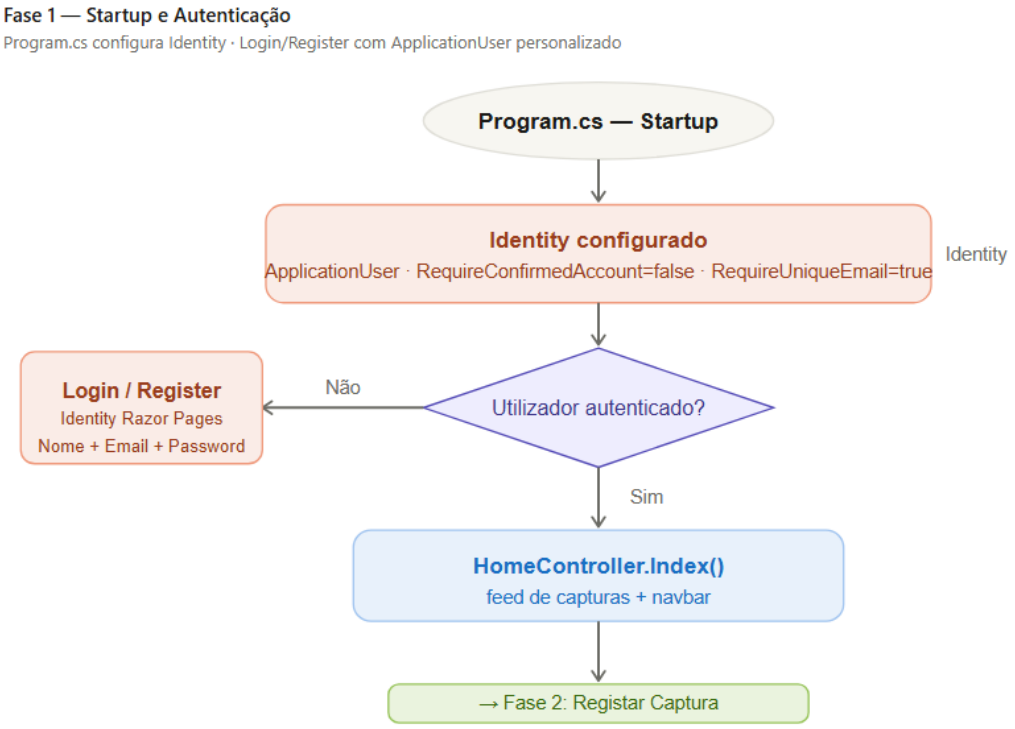
**Principais métodos:**

- AdicionarComentario() — adiciona um comentário a uma captura
- ListarComentarios() — apresenta os comentários de uma captura
- RemoverComentario() — elimina um comentário

```
1   namespace FishCast.Models
2   {
        0 references
3       public class Comentario
4       {
            0 references
5           public int Id { get; set; }
6
            0 references
7           public int CapturaId { get; set; }
8
            0 references
9           public int UtilizadorId { get; set; }
10
            0 references
11          public string? Texto { get; set; }
12
            0 references
13          public DateTime DataComentario { get; set; }
14      }
15  }
```

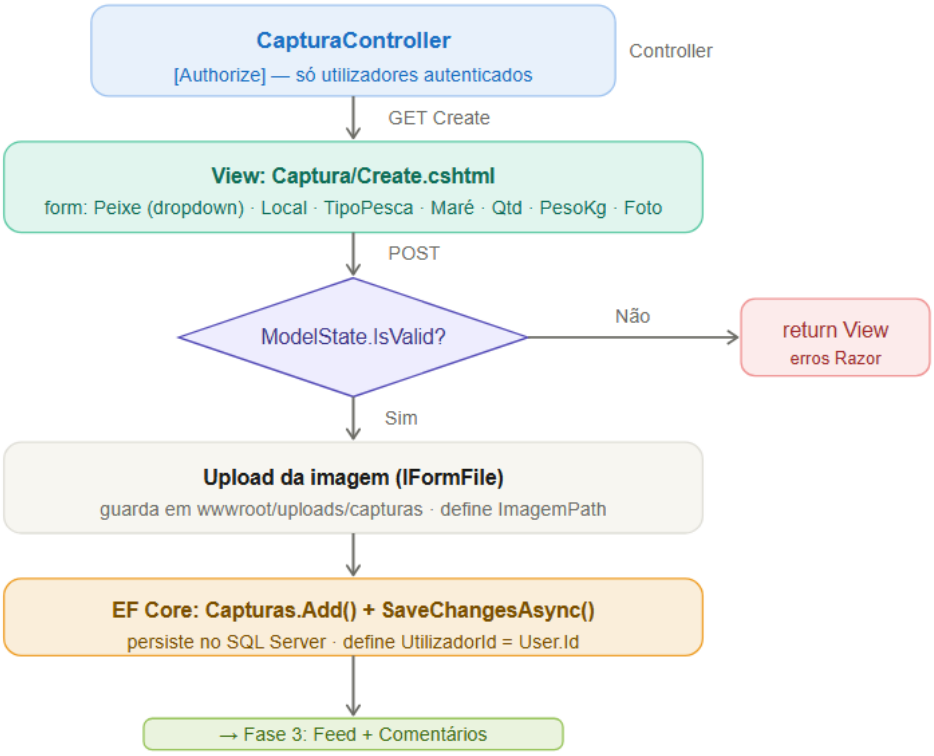
## Outras informações

Fluxograma



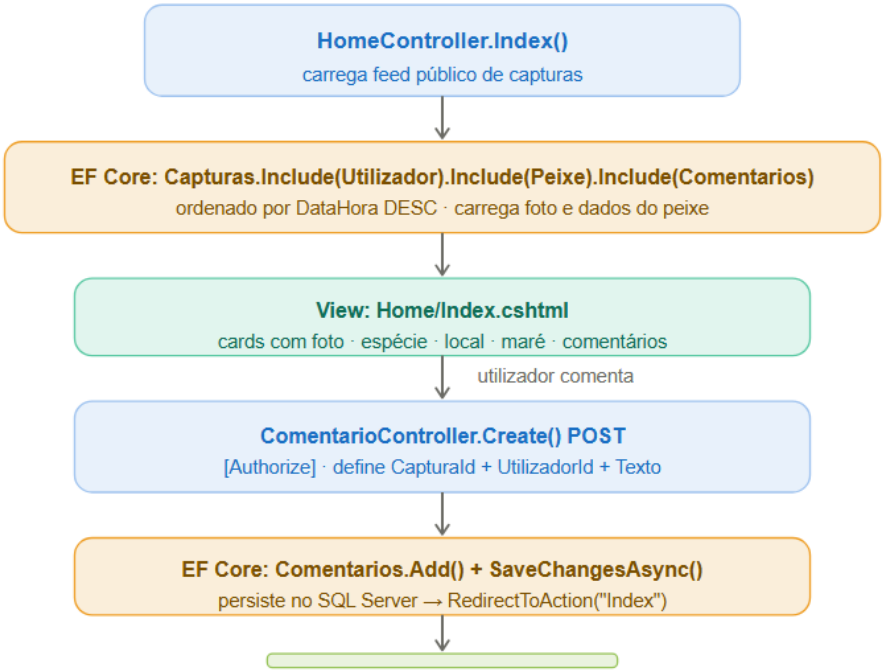
### Fase 2 — Registar Captura

CapturaController [Authorize] · form com foto · ModelState · upload para wwwroot · EF Core INSERT



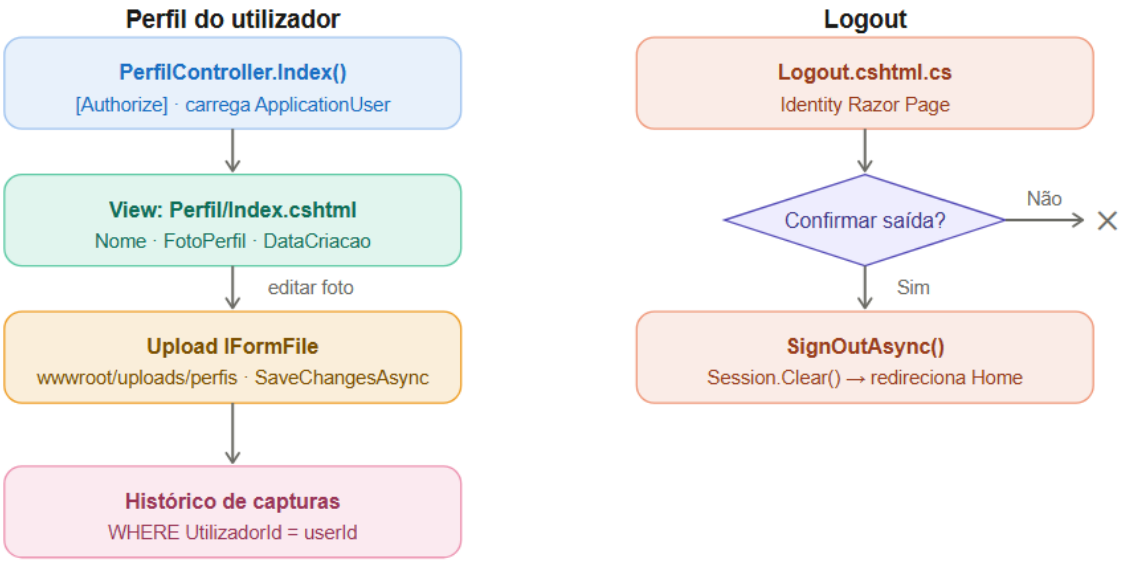

### Fase 3 — Feed e Comentários

Include encadeado · feed público de capturas · ComentarioController com [Authorize]



**Fase 4 — Perfil e Logout**

Perfil com FotoPerfil + histórico · Identity SignOutAsync + Session.Clear()




## Diagrama de Classes

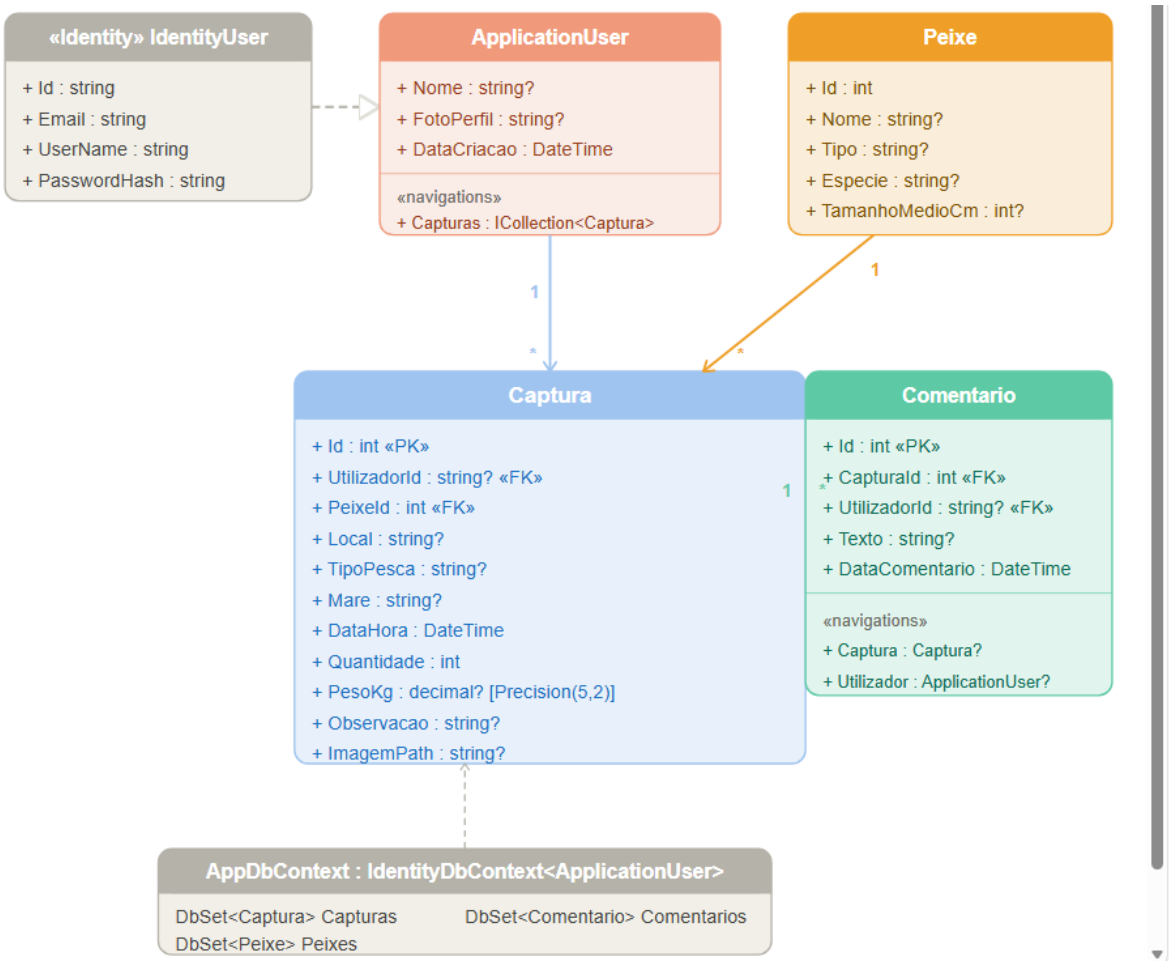
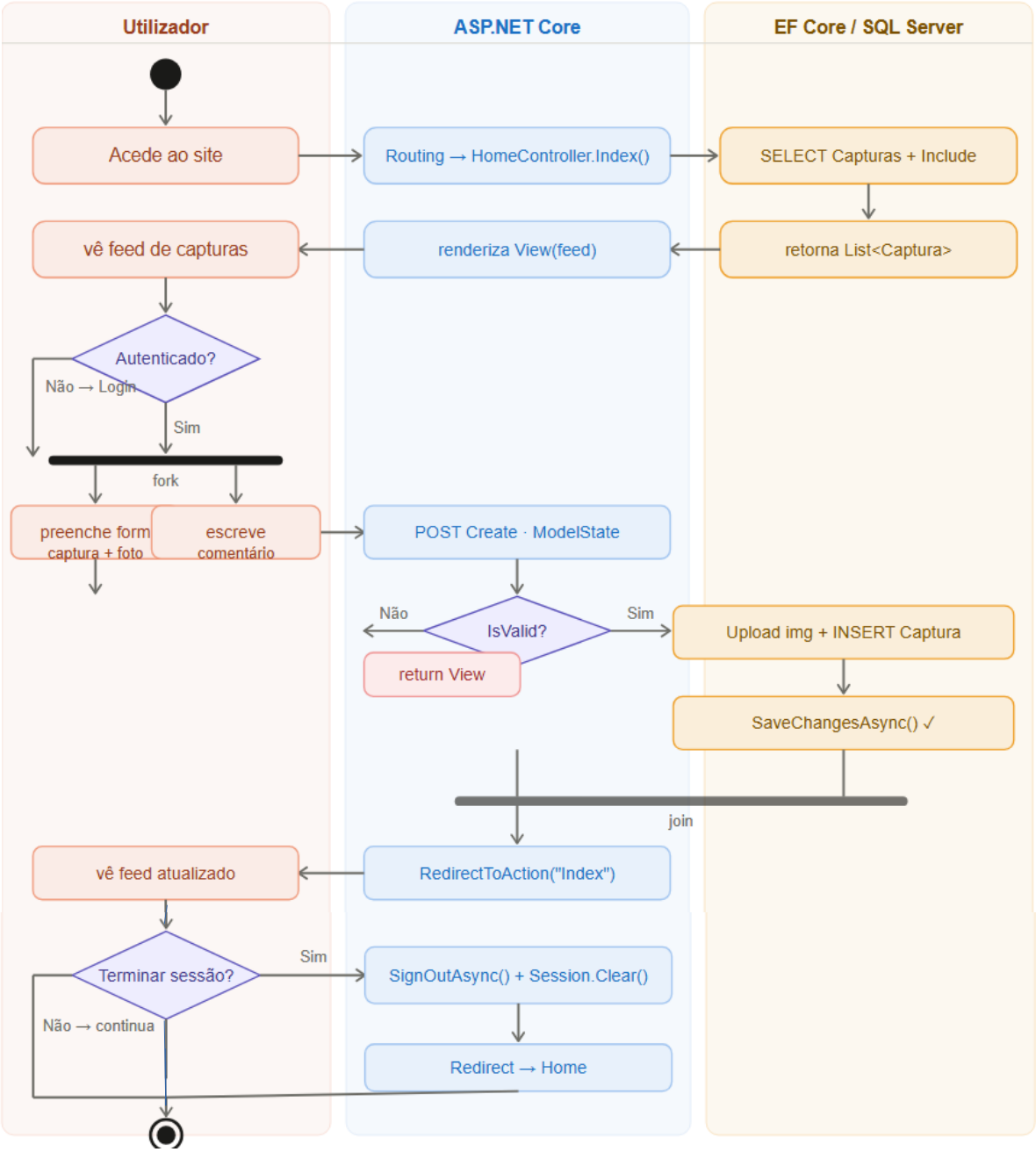



**Diagrama de Atividades**